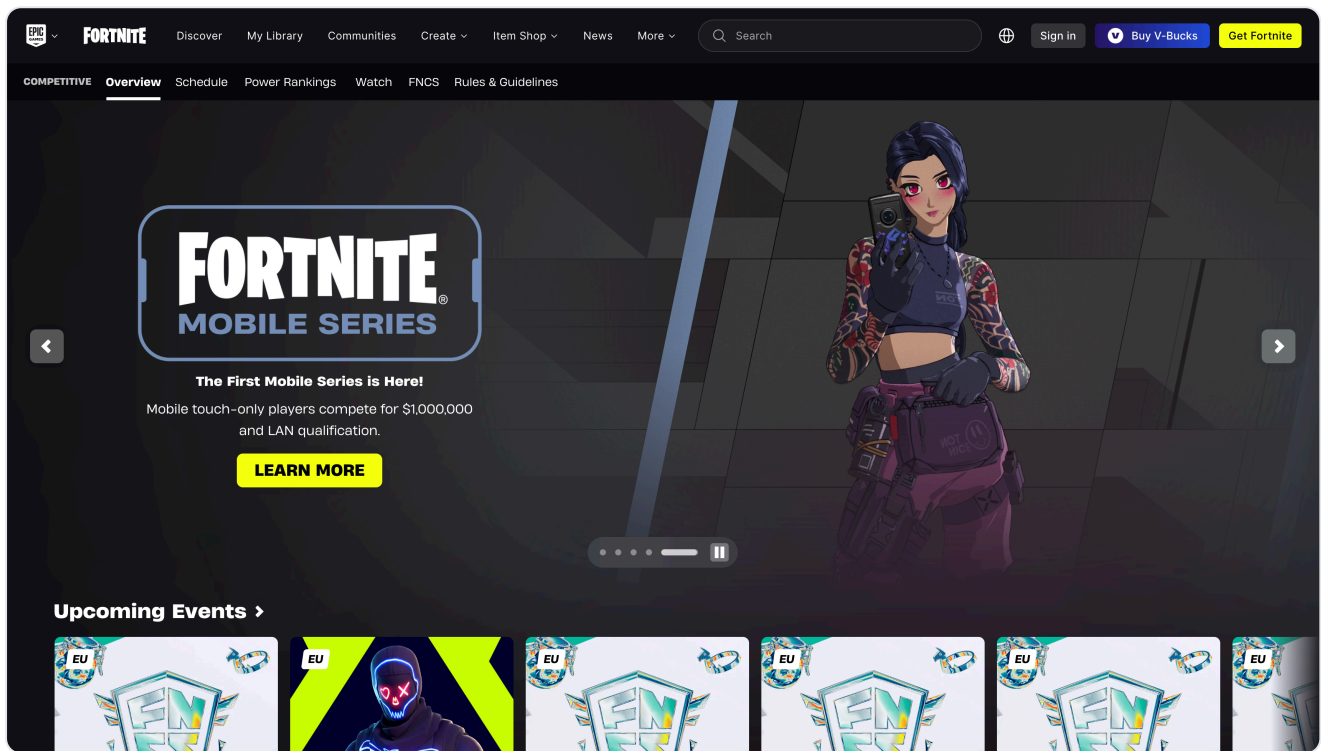


Portfolio: Jan Karlo Dela Cruz

8 years commercial experience in software engineering.

Below are projects that I've been working on recently, both professional and personal.

Fortnite Competitive Web App

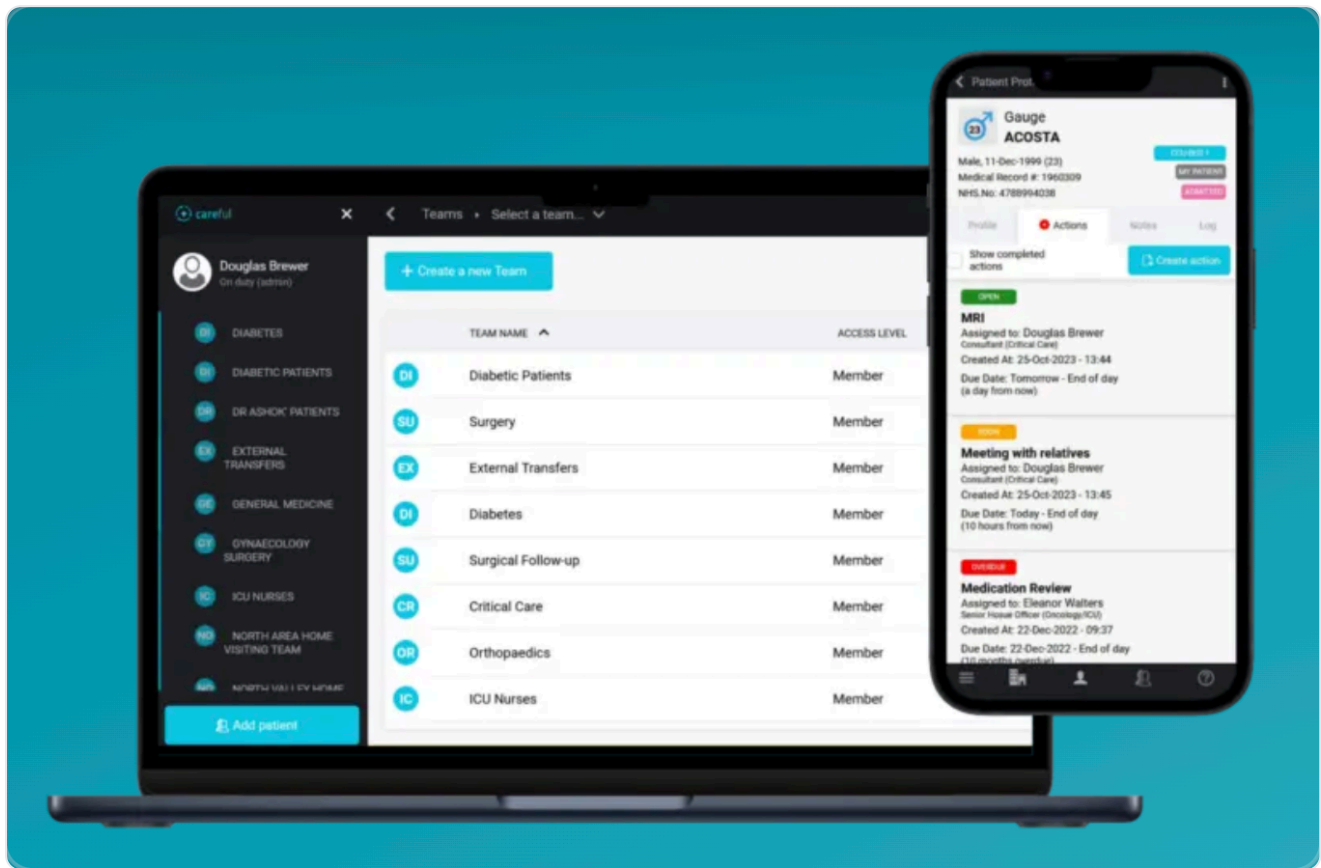


fortnite competitive showcase

I led the production of the <https://fortnite.com/competitive> web app, written in React Router 7.

The interesting technical challenges with this involves dealing with data sources that were made for a Game client. The sizes of data being retrieved is too big to try and fetch things fresh and then displayed to a web client. The solution around it was proactive caching, where it has it's own tradeoffs in strong availability but weak consistency.

Careful Mobile App



Careful web native app

I contributed to sprucing up the UX and UI functionality of this mobile app, written via Capacitor, targeted to Ionic.

The trickiest bits for this role was creating a UI for allowing users to integrate 3rd party data sources; finding balance between flexibility and simplicity.

bote: A FOSS Node.js library, made to stream giant GBs of JSON in low-memory environments

This passion project actually came from the problems from the previous role I had and I also wanted to try agent-assisted development (Claude Code) to see how effective it is.

<https://github.com/jankdc/bote>

A quick glance:

```
import { fileURLToPath } from 'node:url';
import { open, fromFile } from '@botejs/core';

// 181 MB GeoJSON:
// { type: "...", features: [{ properties: { STREET: "..." }}] }
const filePath = fileURLToPath(new URL('../citylots.json', import.meta.url));

await using cursor = await open(fromFile(filePath));

const byStreet = await cursor
  .iter('features', {
    select: ['properties', 'STREET'],
  })
  .reduce((tally, street) => {
    if (typeof street === 'string') {
      tally.set(street, (tally.get(street) ?? 0) + 1);
    }
    return tally;
  }, new Map());

console.log([...byStreet].sort((a, b) => b[1] - a[1]).slice(0, 10));
```

Here's the benchmark that I run on my M1 Macbook:

```

-C/bote-comparison
bote-comparison on P main via v26.3.0
> ./bench.sh
Benchmark 1: JSON.parse
  Time (mean ± σ):    874.4 ms ± 13.3 ms    [User: 905.4 ms, System: 125.4 ms]
  Range (min ... max): 855.2 ms ... 891.5 ms  5 runs

Benchmark 2: bote
  Time (mean ± σ):    484.3 ms ± 7.3 ms     [User: 434.8 ms, System: 106.6 ms]
  Range (min ... max): 474.6 ms ... 494.3 ms  6 runs

Benchmark 3: @streamparser/json
  Time (mean ± σ):    5.087 s ± 0.038 s     [User: 5.051 s, System: 0.236 s]
  Range (min ... max): 5.022 s ... 5.115 s   5 runs

Benchmark 4: JSONStream
  Time (mean ± σ):    4.597 s ± 0.102 s     [User: 4.544 s, System: 0.147 s]
  Range (min ... max): 4.481 s ... 4.727 s   5 runs

Benchmark 5: oboe.js
  Time (mean ± σ):    8.318 s ± 0.448 s     [User: 8.208 s, System: 0.202 s]
  Range (min ... max): 7.904 s ... 8.981 s   5 runs

Benchmark 6: stream-json
  Time (mean ± σ):   12.959 s ± 0.893 s     [User: 13.118 s, System: 0.341 s]
  Range (min ... max): 12.139 s ... 14.458 s  5 runs

Benchmark 7: @discoveryjs/json-ext
  Time (mean ± σ):    1.403 s ± 0.027 s     [User: 1.736 s, System: 0.154 s]
  Range (min ... max): 1.379 s ... 1.445 s   5 runs

Summary
  bote ran
    1.81 ± 0.04 times faster than JSON.parse
    2.90 ± 0.07 times faster than @discoveryjs/json-ext
    9.49 ± 0.25 times faster than JSONStream
   10.50 ± 0.18 times faster than @streamparser/json
   17.17 ± 0.96 times faster than oboe.js
   26.76 ± 1.89 times faster than stream-json

measuring peak memory footprint (5 runs each)
[mem] JSON.parse: 1 2 3 4 5
[mem] bote: 1 2 3 4 5
[mem] @streamparser/json: 1 2 3 4 5
[mem] JSONStream: 1 2 3 4 5
[mem] oboe.js: 1 2 3 4 5
[mem] stream-json: 1 2 3 4 5
[mem] @discoveryjs/json-ext: 1 2 3 4 5

| method | mean time | mean peak footprint (MB) |
| --- | --- | --- |
| bote | 0.484 ± 0.007 s | 46.7 ± 1.1 |
| JSON.parse | 0.874 ± 0.013 s | 938.8 ± 0.4 |
| @discoveryjs/json-ext | 1.403 ± 0.027 s | 403.3 ± 3.3 |
| JSONStream | 4.597 ± 0.102 s | 58.5 ± 0.4 |
| @streamparser/json | 5.087 ± 0.038 s | 65.0 ± 5.6 |
| oboe.js | 8.318 ± 0.448 s | 94.3 ± 0.5 |
| stream-json | 12.959 ± 0.893 s | 158.0 ± 4.1 |

```

bote comparison showcase

It's beaten other popular libraries in the same use-case. I've written this with a combination of TypeScript and Rust, where Rust was used to really optimize the structural position (e.g. `{>[]`) navigation using SIMD.

I'm really proud of this work because I learnt a lot here with memory management, how to profile compute and memory performance as well as debugging memory leaks.